PORA ACADEMY

Data Science

Advanced Python for Data Science

Lambda Function

- A lambda function is a small anonymous function.
- A lambda function can take any number of arguments, but can only have one expression.
- Syntax: `lambda` *`arguments`* `:` *`expression`*
- See example below:

```
x = lambda a, b, c : a + b + c
print(x(5, 6, 2))
```

- The power of lambda is better shown when you use them as an anonymous function inside another function.

Zip Function

- The zip() function returns a zip object, which is an iterator of tuples where the first item in each passed iterator is paired together, and then the second item in each passed iterator are paired together etc.
- If one tuple contains more items, these items are ignored:
- Syntax: zip(iterator1, iterator2, iterator3 ...)

List Comprehension

- List comprehension offers a shorter syntax when you want to create a new list based on the values of an existing list
- Syntax:

  newlist = [expression for item in iterable if condition == True]

- See example below.

```
newlist = [x for x in range(10)]
```

Map, Filter, Reduce

- The map() function executes a specified function for each item in an iterable.
- The filter() function returns an iterator where the items are filtered through a function to test if the item is accepted or not.
- Reduce applies a function of two arguments cumulatively to the elements of an iterable, optionally starting with an initial argument.
- **Map** and **Filter** are Python built-in functions.
- While map can take more than one iterable, filter only accepts one.
- **Reduce** must be imported from the functools library.

Maps Function Syntax: `map(func, *iterables)`
Filter function syntax: `filter(func, iterable)`
Reduce function syntax: `reduce(func, iterable[, initial])`

Classes and Objects

- Python is an object oriented programming language.
- Almost everything in Python is an object, with its properties and methods.
- A Class is like an object constructor, or a "blueprint" for creating objects.
- To create a **class**, use the keyword **class**:
- An **object** is an instance of a class.
- The __init__() function is called automatically every time the class is being used to create a new object.
- See example below.

```
class Person:
  def _init_(self, name, age):
    self.name = name
       self.age = age

p1 = Person("John", 36)

print(p1.name)
print(p1.age)
```

Exception Handling: Try and Expect

- Error in Python can be of two types i.e. Syntax errors and Exceptions.
- **Errors** are problems in a program that will make the program not to execution completely.
- **Exceptions** are raised when some internal events occur which change the normal flow of the program.
- There are several built-in Python exceptions that can be raised when an error occurs during the execution of a program.
- Common examples are; ImportError, ZeroDivisionError, ValueError, IndexError, TypeError, KeyError, AttributeError.
- Exception handling ensures code reliability, easier debugging and simplified error handling.

Exception Handling: Try and Expect

- Try and except statements are used to catch and handle exceptions in Python.
- The **try** block lets you test a block of code for errors.
- The **except** block lets you handle the error.
- The **else** block lets you execute code when there is no error.
- The **finally** block lets you execute code, regardless of the result of the try- and except blocks.
- Syntax:

```
try:
    # Some Code....
except:
    # optional block
    # Handling of exception (if required)
else:
    # execute if no exception
finally:
    # Some code .....(always executed)
```

File Handling

- Python support working with different formats of files (txt, csv, json, tsv.
- File handling in Python allows you to perform a wide range of operations, such as creating, reading, writing, appending, renaming, and deleting files.
- File handling is a fundamental skill for data scientists, enabling them to efficiently store, retrieve, and manipulate data.
- The key function for working with files in Python is the **open()** function
- The open() function takes two parameters; filename, and mode

File Handling: Mode

- There are four different methods (modes) for opening a file:
  - "r" - Read - Default value. Opens a file for reading, error if the file does not exist
  - "a" - Append - Opens a file for appending, creates the file if it does not exist
  - "w" - Write - Opens a file for writing, creates the file if it does not exist
  - "x" - Create - Creates the specified file, returns an error if the file exists
- Additionally, whether text or binary mode can be specified.
  - "t" - Text - Default value. Text mode
  - "b" - Binary - Binary mode (e.g. images)
- Because "r" for read, and "t" for text are the default values, you do not need to specify them.

File Handling: Open and Close

- To open the file, use the built-in open() function.
- The open() function returns a file object, which has a read() method for reading the content of the file

```
f = open("demofile.txt", "r")
print(f.read())
```

- You can read the entire file with **‘rgud’** or read a specific line n by passing number n into the read [**rgud(2]**). **Rgudlings** to read the first line or the file object can be looped over.
- It is a good practice to always close the file when you are done with it.
  f.close()

File Handling: Write

- To write to an existing file, you must add a parameter to the open() function:
- "a" - Append - will append to the end of the file
- "w" - Write - will overwrite any existing content

```
f = open("demofile2.txt", "a")
f.write("Now the file has more content!")
f.close()
#open and read the file after the appending:
f = open("demofile2.txt", "r")
print(f.read())
```

File Handling: Delete

- To delete a file, you must import the OS module, and run its os.remove() function
- To avoid getting an error, you might want to check if the file exists before you try to delete it:

```
import os
if os.path.exists("demofile.txt"):
  os.remove("demofile.txt")
else:
  print("The file does not exist")
```

- To delete an entire folder, use the os.rmdir() method. Only empty folders can be remove.

```
import os
os.rmdir("myfolder")
```

Handling CSV and JSON Files in Python

- The **csv** and **json** module in Python's standard library provides functionality to read from and write to CSV and JSON files respectively.

- Import CSV and use `csv.reader(file)` to read CSV file and `csv.writer(file)` to CSV.
- Import pandas as pd and use `pd.read_csv` to read CSV to Pandas `df.to_csv` dataframe and useto write pandas dataframe to CSV.
- Import JSON and use `json.load(file)` to read json file and `json.dump(file)` to CSV.

- Import pandas as pd and use `pd.read_json` to read json to dataframe and use `df.to_json` to write pandas dataframe to json.

Pandas

Practice Questions

Question 1:
Your task is to calculate the sales commission for a company's sales department. Employees are paid based on the sales they make, and the commission rate varies by region. Apply additional region bonuses to region North. Ensure that missing or invalid data/file is well handled. Save your result into another csv file. The sales data is stored in a CSV file with columns: Employee, Region, SaleAmount, and Date.

Dataset: Download Dataset Here

Practice Questions

Question 2

Build a loan approval system for a bank to evaluate loan applications based on income, credit score, employment status, and loan amount. Approve applications for employed applicants if the credit score is above 670, income is at least 40,000 with a loan amount no more than half of the income, and the loan amount is greater than 50,000. Calculate the interest rate as 0.05 + (700 - credit score) * 0.0001, and flag applications for manual review if they don't meet these criteria.
Save the results to a CSV file with columns for eligibility, interest rate, and high-risk status.

Dataset: Download dataset here

PORA
ACADEMY

THANK YOU!